Planejamento Lógico TaskTracker

Entrega Inicial do Bootcamp II

Identificação	Informação
Nome completo	Nícolas Henrique Tito Lettieri
Curso	Ciência de Dados e Machine Learning
Polo ou turma	Unidade Asa Norte, 2º semestre, matutino
E-mail institucional	nicolas.lettieri@sempreceub.com

Objetivo do documento. Planejar um sistema simples de gerenciamento de tarefas chamado TaskTracker. O documento define o problema, os dados, as regras, os algoritmos em português e a organização futura do repositório. Esse planejamento servirá como referência direta para implementar o sistema em Python nas próximas fases.

1 Identificação e entendimento do problema

1.1 Problema a ser resolvido

Pessoas e equipes frequentemente registram tarefas em papéis, mensagens e arquivos diferentes. Essa dispersão dificulta saber o que precisa ser feito primeiro, quais itens continuam pendentes e quais já foram concluídos. O resultado pode ser atraso, esquecimento e perda de visão sobre o trabalho diário.

1.2 Solução proposta

O TaskTracker será um sistema de linha de comando para centralizar tarefas. O usuário poderá cadastrar uma tarefa com título, descrição, prioridade e data limite; consultar as tarefas organizadas; marcar uma tarefa como concluída; remover tarefas; e visualizar um resumo do andamento. A primeira implementação priorizará clareza, validação dos dados e armazenamento simples em arquivo local.

1.3 Usuário e cenário de uso

O público inicial é uma pessoa que deseja organizar estudos, trabalho ou compromissos pessoais em um único lugar. Exemplo: um estudante cadastra a entrega de uma atividade com prioridade alta, consulta a lista ao iniciar o dia e marca o item como concluído após enviá-lo.

2 Decomposição de dados e regras

2.1 Entradas de dados

Dado	Tipo planejado	Exemplo	Obrigatório
Título	Texto	Entregar atividade	Sim
Descrição	Texto	Revisar PDF e enviar no AVA	Não
Prioridade	Opção	Alta	Sim
Data limite	Data	13/09/2026	Não
Comando do menu	Número	1	Sim

Dado	Tipo planejado	Exemplo	Obrigatório
Identificador da tarefa	Número	3	Quando alterar tarefa

2.2 Saídas de dados

- Lista de tarefas com identificador, título, prioridade, data limite e status.
- Mensagem de confirmação após cadastro, conclusão ou remoção.
- Mensagens de erro para entradas inválidas.
- Resumo com total de tarefas, pendentes e concluídas.
- Aviso quando não houver tarefas cadastradas.

2.3 Estrutura de cada tarefa

Campo	Finalidade
id	Identificar a tarefa sem depender do título.
titulo	Descrever brevemente o que deve ser feito.
descricao	Guardar detalhes opcionais.
prioridade	Classificar como Alta, Média ou Baixa.
data_limite	Registrar o prazo quando existir.
concluida	Indicar se a tarefa está pendente ou concluída.

2.4 Regras do sistema

- O título não pode estar vazio nem conter apenas espaços.
- A prioridade aceita somente Alta, Média ou Baixa.
- Quando informada, a data limite deve seguir o formato DD/MM/AAAA e ser uma data válida.
- Cada tarefa recebe um identificador numérico único.
- Uma tarefa nova começa com o status Pendente.
- Somente uma tarefa existente pode ser concluída ou removida.
- Concluir novamente uma tarefa já concluída não deve duplicar nem corromper dados.
- Alterações devem ser salvas para permanecerem disponíveis depois que o programa fechar.
- Na listagem, tarefas pendentes aparecem antes das concluídas; entre pendentes, prioridade Alta vem antes de Média e Baixa.

3 Receita lógica do sistema

O sistema funcionará por meio de um menu repetido até o usuário escolher sair. Cada opção executará uma função específica e retornará uma mensagem clara.

3.1 Fluxo principal

1. Iniciar o programa.

2. Carregar as tarefas salvas no arquivo local. Se o arquivo ainda não existir, começar com uma lista vazia.

3. Exibir o menu: cadastrar, listar, concluir, remover, mostrar resumo ou sair.

4. Ler a opção escolhida.

5. Executar a função correspondente. Se a opção for inválida, informar o erro.

6. Depois de qualquer alteração, salvar novamente a lista de tarefas.

7. Repetir o menu até o usuário escolher sair.

8. Encerrar o programa com uma mensagem final.

3.2 Cadastro de tarefa

1. Pedir o título e remover espaços extras do início e do fim.

2. Se o título estiver vazio, mostrar erro e pedir novamente.

3. Pedir a descrição, que pode ficar vazia.

4. Pedir a prioridade até receber Alta, Média ou Baixa.

5. Pedir a data limite. Se houver valor, validar formato e existência da data.

6. Gerar um identificador único.

7. Criar a tarefa com status Pendente.

8. Adicionar a tarefa à lista, salvar e confirmar o cadastro.

3.3 Visualização das tarefas

1. Verificar se a lista está vazia. Se estiver, informar que não há tarefas.

2. Ordenar as tarefas: pendentes primeiro; depois por prioridade; por fim, pelo identificador.

3. Exibir uma tarefa por vez com seus principais dados.

4. Representar o status de forma clara como Pendente ou Concluída.

3.4 Conclusão de tarefa

1. Mostrar a lista e pedir o identificador.

2. Validar se o valor é numérico e corresponde a uma tarefa existente.

3. Se a tarefa já estiver concluída, avisar sem alterar os dados.

4. Caso contrário, mudar o status para Concluída, salvar e confirmar.

3.5 Remoção e resumo

Para remover, o sistema pedirá o identificador, validará sua existência, solicitará confirmação e excluirá apenas a tarefa escolhida. Para o resumo, contará o total de tarefas, as pendentes e as concluídas e exibirá os três números.

4 Mapeamento arquitetural para Git e GitHub

4.1 Árvore planejada do repositório

```
tasktracker-projeto/
|-- README.md                # apresentação e instruções
|-- .gitignore               # arquivos temporários ignorados
|-- docs/
|   `-- planejamento-logico.pdf
|-- data/
|   `-- tarefas.json          # dados locais
|-- src/
|   |-- main.py               # menu e fluxo principal
```

```
|   |-- tarefas.py            # regras e operações
|   `-- armazenamento.py      # leitura e gravação do JSON
`-- tests/
    `-- test_tarefas.py       # testes das regras principais
```

4.2 Responsabilidade dos componentes

Componente	Responsabilidade
main.py	Exibir o menu, receber comandos e coordenar o programa.
tarefas.py	Validar dados, cadastrar, ordenar, concluir, remover e resumir tarefas.
armazenamento.py	Carregar e salvar tarefas em formato JSON.
tarefas.json	Manter os dados entre diferentes execuções.
test_tarefas.py	Verificar regras críticas com exemplos automatizados.

4.3 Plano de aprendizado para Git e GitHub

Na Fase 2, pretendo aprender a criar e clonar repositórios, verificar alterações com status, selecionar arquivos, registrar versões com commits claros, criar branches para novas funcionalidades, integrar mudanças, resolver conflitos básicos e publicar o projeto no GitHub. Usarei commits pequenos e descritivos para manter um histórico compreensível.

5 Link da apresentação: https://youtu.be/LwSgzXASZNo